Ruby Keywords Pre 3.0

We don't normally mention deprecated or removed Ruby features in this book, but this is a pattern you are extremely likely to see in older Ruby code.

Before Ruby had true keyword parameters, it had a syntax that simulated them. Any arbitrary key/value pair passed after the positional parameters to a method were automatically rolled up and converted to a Hash. In other words, you could do this:

```
class SongList
  def search(field, options = {})
    # implementation
  end
end

Songlist.new.search(:titles, genre: "jazz", duration_less_than: 270)
```

In that case, the genre and duration_less_than parameters would be rolled together and placed in the last parameter of the method, in this case options (the default empty hash is there in case no extra parameters are passed). It was then the responsibility of the method to determine if the list of key/value pairs in the hash were valid.

Ruby added true keyword parameters in version 2.0, and the true keyword parameters and hash parameters lived awkwardly together until Ruby 3.0, which removed the arbitrary hash parameters, fully replacing them with keyword arguments and the double splat.

Methods and Block Parameters

As we discussed in the section on blocks and iterators on page 62, when a method is called it may be associated with a block. Normally, you call the block from within the method using yield:

```
def double(p1)
  yield(p1 * 2)
end

double(3) { |val| "I got #{val}" }          # => "I got 6"
double("tom") { |val| "Then I got #{val}" } # => "Then I got tomtom"
```

However, if the last parameter in a method definition list is prefixed with an ampersand, any associated block is converted to a Proc object, and that object is assigned to the parameter. This allows you to store the block for use later.

```
class TaxCalculator
  def initialize(name, &block)
    @name, @block = name, block
  end

  def get_tax(amount)
    "#@name on #{amount} = #{ @block.call(amount) }"
  end
end

tc = TaxCalculator.new("Sales tax") { |amt| amt * 0.075 }

tc.get_tax(100) # => "Sales tax on 100 = 7.5"
tc.get_tax(250) # => "Sales tax on 250 = 18.75"
```